

TRAD2026

**TREINAMENTO EM REPRODUTIBILIDADE
PARA ANÁLISE DE DADOS**

Módulo I: Git

Atividade prática



Sumário

1	Comandos Git na prática	3
1.1	Configuração inicial	3
1.2	Criando um repositório local	4
1.3	Criando e adicionando arquivos	4
1.4	Modificando arquivos e criando novos commits	6
1.5	Desfazendo commits com git reset	7
1.6	Trabalhando em paralelo com branches	10
1.7	Identificando e resolvendo conflitos de merge	12
1.8	Ignorando arquivos com .gitignore	17
1.9	Adicionando um repositório remoto	18
1.10	Sincronizando com o repositório remoto	25
2	Projeto prático: usando e colaborando em um repositório remoto	27
2.1	Organização dos grupos	27
2.2	Dados utilizados no projeto	27
2.3	Criando um fork do repositório	28
2.4	Clonando o fork pessoal	28
2.5	Colaborando com o grupo	29

1 Comandos Git na prática

Nesta atividade, você irá explorar os principais comandos do Git e aplicá-los em um repositório local. Siga as instruções para criar arquivos, registrar alterações, consultar o histórico, desfazer commits com segurança, trabalhar com branches, resolver conflitos de merge, ignorar arquivos, configurar o Git para um repositório remoto, e sincronizar alterações com o repositório remoto.

1.1 Configuração inicial

Antes de começar a usar o Git, configure seu nome e endereço de e-mail. Essas informações serão associadas aos commits e permitirão identificar suas contribuições no histórico do repositório.

```
$ git config --global user.name "Seu Nome"
$ git config --global user.email "seu.email@example.com"
```

Dica.

Ao colaborar com outras pessoas, use o mesmo endereço de e-mail associado à sua conta no GitHub ou em outra plataforma de hospedagem de repositórios. Isso permite que seus commits sejam corretamente vinculados ao seu perfil.

Algumas instalações do Git usam **master** como nome padrão da branch principal.

Para definir **main** como nome padrão em novos repositórios, execute:

```
$ git config --global init.defaultBranch main
```

Para listar as configurações do Git, utilize:

```
$ git config --list
user.name=Seu Nome
user.email=seu.email@example.com
init.defaultBranch=main
```

Dica.

O comando `git config --list` pode apresentar outras configurações do sistema e do usuário. Para consultar apenas uma configuração específica, use, por exemplo, `git config --global user.name`.

1.2 Criando um repositório local

O próximo passo é criar um diretório para o seu projeto. Crie um diretório chamado **projeto** e navegue até ele:

```
$ mkdir projeto  
$ cd projeto
```

Dentro do diretório **projeto**, inicie um repositório Git:

```
$ git init  
Initialized empty Git repository in /caminho/para/projeto/.git/
```

Nota.

O comando `git init` inicializa um repositório Git e cria o subdiretório oculto `.git`. Esse subdiretório armazena o histórico, as referências, as configurações locais e outros metadados do repositório.

1.3 Criando e adicionando arquivos

Agora, crie um arquivo chamado **README.md** e adicione um título:

```
$ echo "# Meu Projeto" > README.md
```

Nota.

O operador `>` cria o arquivo **README.md** e adiciona o conteúdo `"# Meu Projeto"`. Se o arquivo já existir, ele será sobrescrito.

Para verificar o estado do repositório e ver quais arquivos foram adicionados ou modificados, use o comando:

```
$ git status  
On branch main
```

```
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)

  README.md

nothing added to commit but untracked files present (use "git add" to track)
```

O arquivo `README.md` aparece como *untracked*, pois ainda não está sendo rastreado pelo Git.

Para adicionar o arquivo `README.md` ao índice (*staging area*), use o comando:

```
$ git add README.md
```

Verifique novamente o estado do repositório:

```
$ git status
On branch main
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   README.md
```

O arquivo está agora no estado *staged* e será incluído no próximo commit.

Agora, você pode criar o primeiro commit para registrar as alterações no repositório. Use o comando:

```
$ git commit -m "Adiciona README.md"
[main (root-commit) f89925b] Adiciona README.md
1 file changed, 1 insertion(+)
create mode 100644 README.md
```

Nota.

Cada commit possui um identificador único, chamado *hash*. No exemplo, o identificador abreviado é `f89925b`. O identificador produzido em seu repositório será diferente.

Consulte o histórico de commits:

```
$ git log
commit f89925becf2e6a17267dadb91e10c9e2391581eb (HEAD -> main)
Author: jvsguerra <jvsguerra@gmail.com>
```

```
Date: Sat Aug 22 13:59:27 2026 -0300
```

```
Adiciona README.md
```

Para visualizar uma versão resumida do histórico, utilize:

```
$ git log --oneline  
f89925b (HEAD -> main) Adiciona README.md
```

1.4 Modificando arquivos e criando novos commits

Agora, vamos modificar o arquivo `README.md` e criar um novo commit. Adicione uma descrição ao arquivo `README.md`:

```
echo "## Descrição do Projeto" >> README.md  
echo "Este é um projeto de exemplo para praticar comandos Git." >> README.md
```

Nota.

O operador `>>` adiciona conteúdo ao final do arquivo sem substituir o conteúdo existente.

Verifique o estado do repositório novamente:

```
$ git status  
On branch main  
Changes not staged for commit:  
  (use "git add <file>..." to update what will be committed)  
  (use "git restore <file>..." to discard changes in working directory)  
        modified: README.md
```

Nota.

O arquivo `README.md` foi modificado, mas as alterações ainda não foram adicionadas à *staging area*. Para descartar as mudanças e restaurar a última versão registrada, use `git restore README.md`.

Consulte as diferenças entre o arquivo atual e a última versão registrada:

```
$ git diff README.md  
diff --git a/README.md b/README.md  
index 160ca7d..08681e8 100644  
--- a/README.md
```

```
+++ b/README.md
@@ -1 +1,3 @@
# Meu Projeto
+## Descrição do Projeto
+Este é um projeto de exemplo para praticar comandos Git.
```

Como o arquivo já está sendo rastreado pelo Git, é possível usar a opção `-a` para adicionar suas modificações à *staging area* e criar o commit em um único comando:

Como foram feitas alterações em um arquivo já existente e rastreado pelo Git, podemos usar a opção `-am` para adicionar as alterações à *staging area* e criar o commit em um único comando:

```
$ git commit -am "Adiciona descrição do projeto"
[main 87a7e21] Adiciona descrição do projeto
1 file changed, 2 insertions(+)
```

Atenção.

A opção `-a` inclui apenas arquivos modificados ou removidos que já estejam sendo rastreados. Arquivos novos ainda precisam ser adicionados com `git add`.

1.5 Desfazendo commits com `git reset`

Aqui, será criado um commit que depois será desfeito. Adicione uma nova seção ao arquivo:

```
$ echo "## Funcionalidades" >> README.md
$ echo "Este projeto possui as seguintes funcionalidades:" >> README.md
$ git commit -am "Adiciona funcionalidades do projeto"
[main d355673] Adiciona funcionalidades do projeto
1 file changed, 2 insertions(+)
```

Consulte o histórico resumido:

```
$ git log --oneline
d355673 (HEAD -> main) Adiciona funcionalidades do projeto
87a7e21 Adiciona descrição do projeto
```

```
f89925b Adiciona README.md
```

O comando `git reset` move o ponteiro da branch para outro commit. O destino `HEAD~1` representa o commit imediatamente anterior ao commit atual.

1.5.1 `git reset --soft`

A opção `--soft` desfaz o commit, mas mantém as alterações na *staging area*:

```
$ git reset --soft HEAD~1
```

Verifique o estado do repositório:

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

As alterações podem ser registradas novamente:

```
$ git commit -m "Adiciona funcionalidades do projeto"
[main 58c06a3] Adiciona funcionalidades do projeto
1 file changed, 2 insertions(+)
```

1.5.2 `git reset --mixed`

A opção `--mixed`, que é o modo padrão do comando, desfaz o commit e remove as alterações da *staging area*. As mudanças permanecem no *working directory*:

```
$ git reset --mixed HEAD~1
Unstaged changes after reset:
M README.md
```

Atenção.

O mesmo resultado pode ser obtido omitindo a opção `--mixed`, pois ela é o modo padrão do comando `git reset`.

Verifique o estado do repositório:


```
$ git status  
On branch main  
Changes not staged for commit:  
  (use "git add <file>..." to update what will be committed)  
      modified: README.md
```

Registre novamente as alterações:

```
$ git commit -am "Adiciona funcionalidades do projeto"  
[main be1402b] Adiciona funcionalidades do projeto  
1 file changed, 2 insertions(+)
```

1.5.3 git reset --hard

A opção `--hard` desfaz o commit e também descarta as alterações correspondentes do *working directory* e da *staging area*:

```
$ git reset --hard HEAD~1  
HEAD is now at 87a7e21 Adiciona descrição do projeto
```

Atenção

Use `git reset --hard` somente quando tiver certeza de que as alterações podem ser descartadas. Esse comando pode provocar perda de trabalho não salvo em outro commit ou arquivo.

Para desfazer um commit que já foi enviado a um repositório compartilhado, prefira `git revert`. Esse comando cria um novo commit que desfaz as alterações anteriores sem reescrever o histórico existente.

Verifique o estado do repositório:

```
$ git status  
On branch main  
nothing to commit, working tree clean
```

Consulte novamente o histórico:

```
$ git log --oneline  
87a7e21 Adiciona descrição do projeto  
f89925b Adiciona README.md
```

1.6 Trabalhando em paralelo com branches

Branches permitem desenvolver funcionalidades, correções ou atualizações de forma isolada, sem modificar diretamente a versão principal do projeto. Neste exemplo, a branch `main` continuará estável enquanto uma nova seção será adicionada ao arquivo `README.md` em uma branch separada.

Antes de começar, verifique a branch atual e o estado do repositório:

```
$ git branch
* main

$ git status
On branch main nothing to commit, working tree clean
```

O asterisco indica a branch atualmente selecionada.

Crie uma branch chamada `adiciona-instalacao` e alterne para ela:

```
$ git switch -c adiciona-instalacao
Switched to a new branch 'adiciona-instalacao'
```

Nota.

A opção `-c` cria uma nova branch e alterna imediatamente para ela. O comando é equivalente a executar `git branch adiciona-instalacao` seguido de `git switch adiciona-instalacao`.

Confirme a branch atual:

```
$ git branch
* adiciona-instalacao
main
```

Adicione uma seção de instalação ao arquivo `README.md`:

```
$ echo "## Instalação" >> README.md
$ echo "Consulte a documentação para instalar o projeto." >> README.md
```

Verifique as alterações:

```
$ git diff README.md
diff --git a/README.md b/README.md
```

```
index 08681e8..8877175 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,5 @@
# Meu Projeto
## Descrição do Projeto
Este é um projeto de exemplo para praticar comandos Git.
+## Instalação
+Consulte a documentação para instalar o projeto.
```

Registre as alterações em um novo commit:

```
$ git add README.md
$ git commit -m "Adiciona instruções de instalação"
[adiciona-instalacao 34c5532] Adiciona instruções de instalação
1 file changed, 2 insertions(+)
```

Visualize o histórico e as branches:

```
$ git log --oneline --graph --decorate
* 34c5532 (HEAD -> adiciona-instalacao) Adiciona instruções de instalação
* 87a7e21 (main) Adiciona descrição do projeto
* f89925b Adiciona README.md
```

Dica.

O comando `git log --oneline --graph --decorate` pode ser salvo com um alias para facilitar o uso com o comando:

```
$ git config --global alias.graph "log --oneline --graph --decorate"
```

Depois, basta digitar `git graph` para visualizar o histórico de forma resumida e gráfica.

A branch `adiciona-instalacao` possui agora um commit que ainda não faz parte da `main`.

1.6.1 Integrando uma branch com merge

Após concluir e revisar a alteração, volte para a branch `main`:

```
$ git switch main
```

```
Switched to branch 'main'
```

Agora, integre a branch `adiciona-instalacao` à `main`:

```
$ git merge adiciona-instalacao
Updating 87a7e21..34c5532
Fast-forward
 README.md | 2 ++
 1 file changed, 2 insertions(+)
```

Nesse caso, o Git realizou um *fast-forward merge*. Como a branch `main` não recebeu novos commits após a criação da branch de trabalho, o Git apenas avançou seu ponteiro até o commit mais recente.

Verifique o histórico:

```
$ git log --oneline --graph --all --decorate
* 34c5532 (HEAD -> main, adiciona-instalacao) Adiciona instruções de instalação
* 87a7e21 Adiciona descrição do projeto
* f89925b Adiciona README.md
```

Depois de confirmar que a integração foi concluída, remova a branch de trabalho:

```
$ git branch -d adiciona-instalacao
Deleted branch adiciona-instalacao.
```

Dica.

Remova uma branch somente depois de verificar que suas alterações foram integradas corretamente. A opção `-d` impede a remoção de uma branch que ainda possui commits não integrados.

1.7 Identificando e resolvendo conflitos de merge

Um conflito de merge pode ocorrer quando branches diferentes modificam a mesma parte de um arquivo. Nesse caso, o Git não consegue decidir automaticamente qual conteúdo deve ser mantido e solicita uma revisão manual.

Nesta atividade, serão criadas duas versões diferentes para a mesma linha do arquivo `README.md`.

1.7.1 Criando uma alteração na branch de trabalho

A partir da `main`, crie uma branch chamada `atualiza-descricao`:

```
$ git switch -c atualiza-descricao  
Switched to a new branch 'atualiza-descricao'
```

Abra o arquivo `README.md` em um editor e altere a linha:

```
Este é um projeto de exemplo para praticar comandos Git.
```

para:

```
Este projeto apresenta uma introdução prática ao Git.
```

Registre a alteração:

```
$ git add README.md  
$ git commit -m "Atualiza descrição do projeto"  
[atualiza-descricao 5d7ce19] Atualiza descrição do projeto  
1 file changed, 1 insertion(+), 1 deletion(-)
```

1.7.2 Criando uma alteração concorrente na main

Volte para a branch `main`:

```
$ git switch main  
Switched to branch 'main'
```

No arquivo `README.md`, altere a mesma linha para um conteúdo diferente:

```
Este projeto contém exemplos de comandos para controle de versão.
```

Registre a alteração na `main`:

```
$ git add README.md  
$ git commit -m "Reformula descrição do projeto"  
[main 6e43a9a] Reformula descrição do projeto  
1 file changed, 1 insertion(+), 1 deletion(-)
```

As branches possuem agora alterações diferentes na mesma linha do arquivo.

1.7.3 Provocando o conflito

Tente integrar a branch `atualiza-descricao` à `main`:

```
$ git merge atualiza-descricao
Auto-merging README.md CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Verifique o estado do repositório:

```
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified: README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

O arquivo `README.md` apresentará marcadores semelhantes aos seguintes:

```
$ git diff README.md
diff --cc README.md
index 57c7db0,2b5d4f2..0000000
--- a/README.md
+++ b/README.md
@@@ -1,5 -1,5 +1,9 @@@
  # Meu Projeto
  ## Descrição do Projeto
+<<<<<<< HEAD
+Este projeto contém exemplos de comandos para controle de versão.
+=====
+ Este projeto apresenta uma introdução prática ao Git.
++>>>>>>> atualiza-descricao
  ## Instalação
  Consulte a documentação para instalar o projeto.
```

Atenção.

O Git não consegue decidir qual versão deve ser mantida, pois ambas as branches modificaram a mesma linha do arquivo. É necessário revisar o conteúdo e escolher a versão desejada ou combinar as alterações. O arquivo não pode ser registrado até que o conflito seja resolvido.

- <<<<<< HEAD: início da versão presente na branch atual, neste caso, main;
- =====: separação entre as duas versões;
- >>>>>> atualiza-descricao: fim da versão proveniente da outra branch.

1.7.4 Resolvendo o conflito

Abra o arquivo `README.md` em um editor, escolha o conteúdo desejado ou combine as duas versões. Por exemplo:

Este projeto apresenta exemplos práticos de Git e controle de versão.

Remova todos os marcadores de conflito e salve o arquivo. Em seguida, revise a diferença resultante:

```
$ git diff README.md
diff --cc README.md
index 57c7db0,2b5d4f2..0000000
--- a/README.md
+++ b/README.md
@@@ -1,5 -1,5 +1,9 @@@
  # Meu Projeto
  ## Descrição do Projeto
- Este projeto contém exemplos de comandos para controle de versão.
- Este projeto apresenta uma introdução prática ao Git.
++Este projeto apresenta exemplos práticos de Git e controle de versão.
  ## Instalação
  Consulte a documentação para instalar o projeto.
```

Marque o conflito como resolvido:

```
$ git add README.md
```

Finalize o merge com um commit:

```
$ git commit -m "Resolve conflito na descrição do projeto"
[main 03d100b] Resolve conflito na descrição do projeto
```

Verifique o histórico completo:

```
$ git log --oneline --graph --all --decorate
* 03d100b (HEAD -> main) Resolve conflito na descrição do projeto
|\
| * 5d7ce19 (atualiza-descricao) Atualiza descrição do projeto
* | 6e43a9a Reformula descrição do projeto
|/
* 34c5532 Adiciona instruções de instalação
* 87a7e21 Adiciona descrição do projeto
* f89925b Adiciona README.md
```

Depois de confirmar que o merge foi concluído corretamente, remova a branch:

```
$ git branch -d atualiza-descricao
Deleted branch atualiza-descricao.
```

Atenção.

Um conflito de merge não significa que o repositório foi corrompido. O conflito indica apenas que o Git precisa de uma decisão humana para definir o conteúdo final. Antes de concluir o merge, revise o arquivo, remova todos os marcadores de conflito e teste o resultado.

1.7.5 Cancelando um merge

Se você não quiser continuar a resolução do conflito, pode cancelar o merge e retornar ao estado anterior:

```
$ git merge --abort
```

Esse comando deve ser executado antes de concluir o merge com um commit.

1.8 Ignorando arquivos com .gitignore

Nem todos os arquivos de um projeto devem ser versionados. Arquivos temporários, resultados gerados automaticamente, dados grandes, credenciais e configurações locais devem permanecer fora do repositório.

Para definir quais arquivos o Git deve ignorar, crie um arquivo chamado `.gitignore` na raiz do repositório:

```
$ echo "*.log" > .gitignore
```

Crie um arquivo de exemplo (`run.log`) e consulte o estado do repositório:

```
$ echo "Registro temporário" > run.log
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore

nothing added to commit but untracked files present (use "git add" to track)
```

O arquivo `run.log` não será exibido entre os arquivos não rastreados porque corresponde ao padrão `*.log` definido no `.gitignore`.

O próprio arquivo `.gitignore` deve ser versionado para que as regras sejam compartilhadas com os demais colaboradores:

```
$ git add .gitignore
$ git commit -m "Adiciona regras ao gitignore"
[main d47f9e0] Adiciona regras ao gitignore
1 file changed, 1 insertion(+)
create mode 100644 .gitignore
```

Atenção.

O arquivo `.gitignore` afeta arquivos que ainda não estão sendo rastreados. Adicionar uma regra ao `.gitignore` não remove automaticamente um arquivo que já faz parte do histórico do Git.

Caso um arquivo tenha sido adicionado ao Git por engano, remova-o apenas do

índice, mantendo a cópia local:

```
$ git rm --cached <arquivo>
$ git commit -m "Remove <arquivo> de log do versionamento"
```

Para obter exemplos de arquivos `.gitignore` para diferentes linguagens e frameworks, consulte o repositório oficial do GitHub: <https://github.com/github/gitignore>

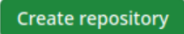
1.9 Adicionando um repositório remoto

Até este ponto, todas as operações foram realizadas no repositório local. Para compartilhar o projeto e colaborar com outras pessoas, é necessário vinculá-lo a um repositório remoto hospedado em uma plataforma como GitHub, GitLab ou Bitbucket.

Nesta atividade, será utilizado o [GitHub](#).

1.9.1 Criando o repositório no GitHub

Acesse o [GitHub](#) e crie um novo repositório:

1. clique em  **New** ;
2. informe um nome para o repositório, como `projeto`;
3. descreva o repositório (opcional);
4. escolha se o repositório será público ou privado;
5. não adicione `README.md`, `.gitignore` ou licença;
6. clique em  **Create repository** ;

Atenção.

Como o repositório local já possui commits, crie um repositório remoto vazio. A criação de arquivos diretamente no GitHub pode gerar históricos diferentes e exigir uma integração adicional antes do primeiro envio.

1.9.2 Associando o repositório local ao remoto

Copie a URL SSH exibida pelo GitHub. Ela terá um formato semelhante a:

```
git@github.com:seu-usuario/projeto.git
```

No terminal, dentro do diretório `projeto`, associe a URL ao nome `origin`:

```
$ git remote add origin git@github.com:seu-usuario/projeto.git
```

Nota.

O nome `origin` é uma convenção utilizada para identificar o repositório remoto principal. Outros nomes podem ser usados, mas `origin` é o padrão adotado pelo Git ao clonar um repositório.

Verifique os repositórios remotos configurados:

```
$ git remote -v
origin git@github.com:seu-usuario/projeto.git (fetch)
origin git@github.com:seu-usuario/projeto.git (push)
```

As duas linhas indicam as URLs usadas para baixar e enviar alterações.

Para consultar informações adicionais sobre o remoto, execute:

```
$ git remote show origin
* remote origin
Fetch URL: git@github.com:jvsguerra/projeto.git
Push URL: git@github.com:jvsguerra/projeto.git
HEAD branch: (unknown)
```

1.9.3 Enviando a branch principal

Envie a branch `main` para o repositório remoto:

```
$ git push -u origin main
```

Nota.

A opção `-u`, abreviação de `--set-upstream`, estabelece uma associação entre a branch local `main` e a branch remota `origin/main`.

Após essa configuração inicial, os próximos envios poderão ser feitos com:

```
$ git push
```

Acesse o repositório no GitHub e confirme que o arquivo `README.md` e o histórico de commits foram enviados.

Dica

Se a autenticação SSH estiver configurada corretamente, o GitHub reconhecerá sua chave durante o primeiro `push`. Para testar a conexão antes do envio, utilize:

```
$ ssh -T git@github.com  
Hi <seu-usuario>! You've successfully authenticated, but GitHub does not  
provide shell access.
```

Caso a autenticação não esteja configurada, acesse a documentação oficial do GitHub:

<https://docs.github.com/pt/authentication/connecting-to-github-with-ssh>.

1.9.4 Enviando uma branch de trabalho

Crie uma nova branch para atualizar a documentação:

```
$ git switch -c atualiza-documentacao  
Switched to a new branch 'atualiza-documentacao'
```

Adicione uma nova seção ao arquivo `README.md`:

```
$ echo "## Documentação" >> README.md  
$ echo "Consulte os materiais do projeto para mais informações." >> README.md
```

Registre a alteração:

```
$ git add README.md  
$ git commit -m "Atualiza documentação do projeto"  
[atualiza-documentacao 70f24f4] Atualiza documentação do projeto  
1 file changed, 2 insertions(+)
```

Envie a nova branch para o repositório remoto:

```
$ git push -u origin atualiza-documentacao  
Enumerating objects: 5, done.  
Counting objects: 100% (5/5), done.
```

```
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 394 bytes | 394.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'atualiza-documentacao' on GitHub by visiting
remote: https://github.com/seu-usuario/projeto/pull/new/atualiza-documentacao
remote:
To github.com:seu-usuario/projeto.git
 * [new branch] atualiza-documentacao -> atualiza-documentacao
branch 'atualiza-documentacao' set up to track 'origin/atualiza-documentacao'.
```

Dica.

O GitHub exibirá a URL para abrir um *Pull Request* e propor a integração da branch enviada à branch `main`. Copie e cole a URL no navegador para abrir o *Pull Request*.

1.9.5 Abrindo um Pull Request

Após enviar a branch para o repositório remoto, abra um *Pull Request* para propor a integração das mudanças à branch de destino.

Quando uma branch tiver sido enviada, use o link exibido no terminal, acesse a aba **Pull requests** do repositório no GitHub e clique em **New pull request**, ou o GitHub exibirá o botão **Compare & pull request**, para abrir um novo *Pull Request*.

atualiza-documentacao had recent pushes 10 minutes ago

Compare & pull request

Na página de criação do *Pull Request*, confirme as branches selecionadas:

base: main ← compare: atualiza-documentacao ✓ Able to merge. These branches can be automatically merged.

Dica.

A branch de destino (`base:`) é a branch que receberá as alterações, geralmente a branch `main`. A branch de origem (`compare`) é a branch que contém as alterações a serem integradas.

Por fim, clique em [Create pull request](#). A contribuição ficará disponível para discussão, revisão e aprovação antes do merge.

1.9.6 Atualizando o Pull Request

Caso a revisão solicite alterações, volte à branch de trabalho:

```
$ git switch atualiza-documentacao
```

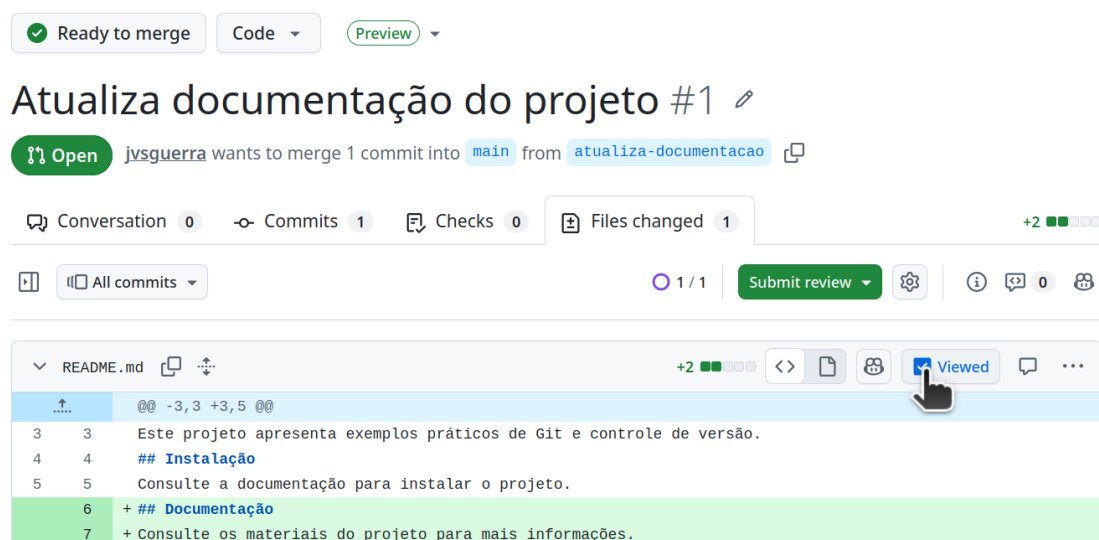
Edite o arquivo, registre as correções e envie novos commits:

```
$ git add README.md  
$ git commit -m "Ajusta documentação após revisão"  
$ git push
```

O *Pull Request* será atualizado automaticamente com os novos commits. Não é necessário abrir outro *Pull Request* para a mesma branch.

1.9.7 Integrando um Pull Request

Antes de integrar o *Pull Request*, revise as alterações em *Files changed*, teste as mudanças e confirme se não há conflitos.



Ready to merge Code Preview

Atualiza documentação do projeto #1

Open jvsguerra wants to merge 1 commit into main from atualiza-documentacao

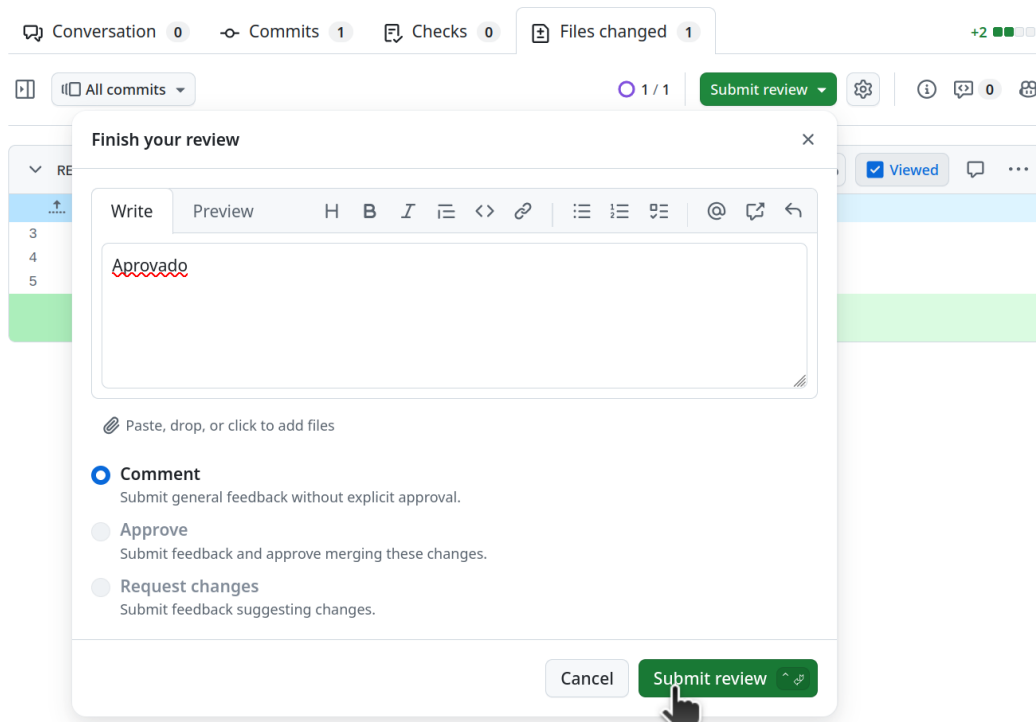
Conversation 0 Commits 1 Checks 0 Files changed 1 +2

All commits 1 / 1 Submit review

README.md +2

↑	@@ -3,3 +3,5 @@
3	Este projeto apresenta exemplos práticos de Git e controle de versão.
4	## Instalação
5	Consulte a documentação para instalar o projeto.
6	+ ## Documentação
7	+ Consulte os materiais do projeto para mais informações.

Depois que as mudanças forem revisadas, o revisor deve aprovar as mudanças ou solicitar ajustes.



Quando o *Pull Request* estiver pronto para ser integrado, clique em

Merge pull request

e confirme a integração.



Pull request successfully merged and closed
You're all set — the branch has been merged.

Nota.

O GitHub oferece diferentes estratégias para integrar um *Pull Request*. A opção *Create a merge commit* preserva os commits da branch de origem e adiciona um commit de merge, mantendo explícito o ponto de integração no histórico. A opção *Squash and merge* reúne todas as alterações do Pull Request em um único commit na branch de destino, o que pode facilitar a leitura do histórico quando existem vários commits intermediários. Por fim, a opção *Rebase and merge* reaplica individualmente os commits da branch de origem sobre a branch de destino, sem criar um commit de merge, produzindo um histórico linear.

Após o merge, atualize a branch **main** local:

```
$ git switch main
$ git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```



```
Unpacking objects: 100% (1/1), 921 bytes | 921.00 KiB/s, done.
From github.com:jvsguerra/projeto
* branch main -> FETCH_HEAD
d47f9e0..668bf92 main -> origin/main
Updating d47f9e0..668bf92
Fast-forward
 README.md | 2 ++
 1 file changed, 2 insertions(+)
```

Quando a branch de trabalho não for mais necessária, remova-a localmente:

```
$ git branch -d atualiza-documentacao
Deleted branch atualiza-documentacao (was 70f24f4).
```

Para remover a branch remota pelo terminal, utilize:

```
$ git push origin --delete atualiza-documentacao
To github.com:jvsguerra/projeto.git
- [deleted] atualiza-documentacao
```

Dica.

A branch remota também pode ser excluída pela interface do GitHub após o merge.

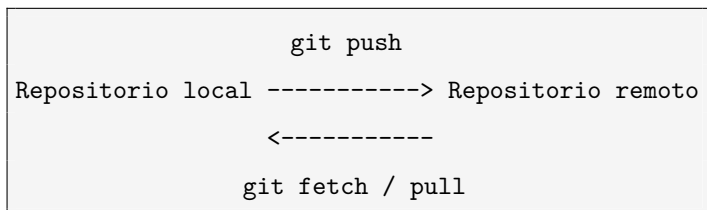
1.10 Sincronizando com o repositório remoto

Durante um trabalho colaborativo, o repositório remoto pode receber novos commits e branches. Por isso, é importante sincronizar o repositório local antes de iniciar uma nova tarefa.

Os principais comandos de sincronização são `git fetch`, `git pull` e `git push`:

- **git fetch**: baixa informações sobre commits e branches do repositório remoto, sem alterar automaticamente os arquivos locais;
- **git pull**: baixa e integra as alterações da branch remota à branch local atual;
- **git push**: envia commits e branches locais para o repositório remoto.

O fluxo pode ser representado da seguinte forma:



2 Projeto prático: usando e colaborando em um repositório remoto

Neste projeto, os participantes aplicarão os conceitos apresentados no módulo para executar uma análise de controle de qualidade e colaborar em um repositório remoto.

Os participantes serão organizados em grupos. Cada grupo deverá executar os scripts disponíveis no repositório do treinamento, interpretar os resultados e preencher um questionário em Markdown. As contribuições individuais serão integradas ao branch do grupo por meio de *Pull Requests*.

2.1 Organização dos grupos

Cada grupo será associado a um branch específico no repositório do treinamento:

```
group1  
group2  
group3  
...  
groupX
```

Atenção.

Os participantes não possuem acesso de escrita ao repositório [cnpem/TRAD2026](https://github.com/cnpem/TRAD2026). Portanto, cada participante deverá trabalhar em um fork pessoal e enviar sua contribuição por meio de um *Pull Request*. A nomenclatura recomendada para o branch no fork do participante é:

```
groupX-seu-usuario
```

2.2 Dados utilizados no projeto

Os dados utilizados no projeto estão descritos no seguinte registro do Zenodo:

<https://zenodo.org/records/22071921>.

Consulte as orientações fornecidas pelos instrutores para identificar o conjunto

de dados atribuído ao grupo e a localização dos arquivos disponibilizados para a atividade.

Atenção.

Não adicione dados brutos, arquivos de sequenciamento ou resultados grandes ao repositório Git. Esses arquivos devem permanecer no local indicado pelos instrutores e ser ignorados por meio do arquivo `.gitignore`.

2.3 Criando um fork do repositório

Acesse o repositório do treinamento no GitHub: <https://github.com/cnpem/TRAD2026>.

Clique em  e crie uma cópia do repositório em sua conta pessoal.

Depois da criação, o fork estará disponível em um endereço semelhante a:

<https://github.com/seu-usuario/TRAD2026>.

Nota.

Um fork é uma cópia de um repositório criada na conta do participante. As alterações realizadas no fork não modificam diretamente o repositório original da organização.

2.4 Clonando o fork pessoal

Clone o fork criado em sua conta:

```
$ git clone git@github.com:seu-usuario/TRAD2026.git  
$ cd TRAD2026
```

O Git configura automaticamente o fork pessoal como o repositório remoto origin.

Confirme a configuração:

```
$ git remote -v  
(fetch) origin git@github.com:seu-usuario/TRAD2026.git  
(push) origin git@github.com:seu-usuario/TRAD2026.git
```

Agora, crie sua branch individual:

```
$ git switch -c groupX-seu-usuario
```

2.5 Colaborando com o grupo

Os scripts da atividade estão disponíveis no diretório: `TRAD2026/git-module/scripts/`

Acesse o diretório:

```
$ cd git-module/scripts  
$ ls
```

Antes de executar um script, consulte suas instruções em `INSTRUCTIONS.md`.

```
$ cat INSTRUCTIONS.md
```

Siga as orientações para identificar os arquivos de entrada, os parâmetros necessários e os resultados esperados.

Após a execução dos scripts, analise os resultados do controle de qualidade e divida as questões do questionário entre os integrantes do grupo. Cada participante deverá preencher as informações solicitadas no cabeçalho e responder a questão atribuída no arquivo `QUESTIONARIO.md`, disponível no diretório `TRAD2026/git-module/`.

Retorne ao diretório do módulo e abra o questionário em um editor:

```
$ cd ..  
$ notepad QUESTIONARIO.md # Windows  
$ nano QUESTIONARIO.md # Linux
```

Dica.

Edite somente as informações do cabeçalho solicitadas e a questão atribuída a você. Não modifique as respostas sob responsabilidade dos demais integrantes.

Após preencher o questionário, revise as alterações:

```
$ git status  
$ git diff QUESTIONARIO.md
```

Confirme que somente as alterações esperadas estão presentes. Em seguida, registre a contribuição na branch individual:

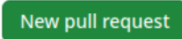
```
$ git add QUESTIONARIO.md  
$ git commit -m "Responde pergunta N do questionário"
```

Envie a branch individual para o fork pessoal:

```
$ git push -u origin groupX-seu-usuario
```

2.5.1 Abrindo o Pull Request

Agora, abra um *Pull Request* para propor a integração da sua contribuição à branch do grupo.

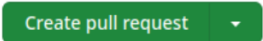
Acesse a aba *Pull Requests* do repositório [cnpem/TRAD2026](#) e clique em .

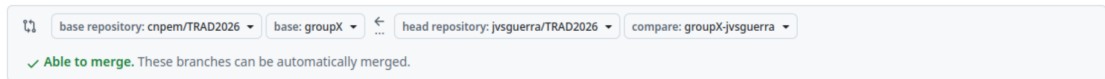
Na página de criação do *Pull Request*, branches de forks podem não aparecer inicialmente entre as opções. Nesse caso, clique em *compare across forks* para exibir os repositórios disponíveis.

Compare changes

Compare changes across branches, commits, tags, and more below. If you need to, you can also [compare across forks](#).

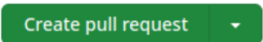
Selecione o seu fork como branch de origem (`seu-usuario:groupX-seu-usuario`) e a branch do grupo como branch de destino (`cnpem:groupX`). Então, clique em

.



Informe um título que identifique claramente a contribuição. Por exemplo:

Responde pergunta 2 do questionario de QC

Na descrição do *Pull Request*, informe o conjunto de dados analisado e a questão respondida. Então, clique em  para abrir o *Pull Request*.

2.5.2 Revisando as contribuições

Cada *Pull Request* deverá ser revisado por pelo menos outro integrante do grupo.

Como o repositório é público, os participantes podem visualizar as mudanças, comentar linhas específicas, sugerir correções e registrar uma revisão.

Durante a revisão:

- confira se a resposta corresponde à questão atribuída;
- verifique se a interpretação está de acordo com os resultados;
- revise a clareza e a formatação do texto;
- confirme que nenhum dado bruto ou resultado grande foi incluído;
- faça comentários ou sugira alterações quando necessário;
- registre a revisão quando a contribuição estiver adequada.

Caso sejam solicitadas alterações, o autor deverá modificar o arquivo na mesma branch, criar um novo commit e enviar as correções:

```
$ git add QUESTIONARIO.md  
$ git commit -m "Ajusta resposta apos revisao"  
$ git push
```

O *Pull Request* será atualizado automaticamente.

Atenção.

Os participantes podem revisar e comentar *Pull Requests*, mas não possuem permissão para integrar alterações ao repositório `cnpem/TRAD2026`. O merge será realizado por um instrutor ou monitor com permissão de escrita.

Dica.

As contribuições devem ser integradas à branch do grupo uma por vez.

Para acompanhar as atualizações do repositório institucional, configure o repositório `cnpem/TRAD2026` como remoto `upstream`.

Essa configuração precisa ser realizada apenas uma vez, após clonar o fork pessoal:

```
$ git remote add upstream git@github.com:cnpem/TRAD2026.git
```

Verifique os repositórios remotos configurados:

```
$ git remote -v
(fetch) origin git@github.com:seu-usuario/TRAD2026.git
(push) origin git@github.com:seu-usuario/TRAD2026.git
(fetch) upstream git@github.com:cnpem/TRAD2026.git
(push) upstream git@github.com:cnpem/TRAD2026.git
```

Nesse fluxo, `origin` representa o fork pessoal, para o qual o participante possui permissão de escrita, enquanto `upstream` representa o repositório institucional, utilizado como fonte das atualizações oficiais.

Depois de cada merge realizado na branch `groupX`, atualize as referências do repositório institucional e integre as alterações à sua branch individual:

```
$ git fetch upstream
$ git switch groupX-seu-usuario
$ git merge upstream/groupX
```

Caso a atualização gere conflitos, revise o arquivo `QUESTIONARIO.md`, preserve as respostas já integradas e resolva os conflitos antes de continuar.

Depois da resolução, registre e envie as alterações para a branch no fork pessoal:

```
$ git add QUESTIONARIO.md
$ git commit -m "Atualiza questionario com contribuicoes do grupo"
$ git push
```

O *Pull Request* será atualizado automaticamente. Esse processo reduz a ocorrência de conflitos e garante que cada contribuição seja integrada sobre a versão mais recente da branch `groupX`. *Pull Request*.